Event_Data_Diagnostics.m: Documentation

Last updated October 21, 2026. Companion documents: Event_Data_Stream_Simulation_v3.m Documentation and Event Simulator Calibration Plan.

1. Purpose

This script measures an event file and prints the numbers you need to calibrate the simulator. The idea is to run the same script on two files: the real recording and the synthetic file. The real run gives your calibration targets, and you tune the simulator until the synthetic run reproduces the same numbers and plots.

It is a plain script with no function files. It does not find the target on its own: you give it the track as a straight line, taken from the simulator's printout (synthetic data) or from your angular-rate pipeline's fit (real data).

2. Quick start

1. Set `fname`, `nx`, `ny` and `zeroIndexed` for your file.
2. Fill in the track (`trackX0`, `trackY0`, `trackVx`, `trackVy`, `trackT0`). For synthetic data, copy the line the simulator prints at the end of its run.
3. Run. It prints a calibration summary and opens three figures.
4. Repeat on the other file and compare the two summaries.

It needs R2019a or newer (`readmatrix`) and base MATLAB only.

3. Input reference

Variable	Default	Units	What it does
`fname`	'synthetic_debris_events.txt'	none	Event file with columns x, y, polarity, t_us.
`dataLabel`	'synthetic'	none	Name shown in plot titles and the console header.

Variable	Default	Units	What it does
`nx`, `ny`	1280, 720	px	Sensor size. The script stops with an error if any event falls outside it.
`zeroIndexed`	false	none	true if the file's x and y start at 0 (common in raw Prophesee exports); the script then adds 1.
`tZero_us`	[]	us	What counts as t = 0. Empty uses the file's first timestamp; 0 keeps raw timestamps. Must match the time zero used for the track.
`trackX0`, `trackY0`	1280, 90	px	Track position at time `trackT0`.
`trackVx`, `trackVy`	-612.44, 43.06	px/s	Track velocity.
`trackT0`	0	s	Time at which the track is at (X0, Y0).
`tc`	[]	s	Center time of the windowed snapshot. Empty uses the middle of the file.
`winLen`	5e-3	s	Width of the snapshot window.
`netClim`	10	events	Snapshot color range, plus or minus this many net events per pixel.
`zoomX`, `zoomY`	70, 35	px	Half-width and half-height of the snapshot zoom.
`nTb`	200	bins	Number of time bins in the event-rate plot.
`alongRange`	[-300 30]	px	Along-track range of the gate. Negative is behind the target.

Variable	Default	Units	What it does
`crossGate`	15	px	Gate half-width either side of the track.
`bgExclude`	40	px	Pixels closer than this to the track are ignored when measuring background.

4. Defining the track

The track is a straight line in pixels and seconds: `x(t) = trackX0 + trackVx * (t - trackT0)`, and the same for y.

Synthetic file. The simulator prints a ready-made line, for example `trackX0 = 1280; trackY0 = 90; trackVx = -612.44; trackVy = 43.06; trackT0 = 0;`. Paste those values in. Set `tZero_us = []` or 0; both are fine because the simulator's times already start near 0.

Real file. Take the values from your angular-rate pipeline's fit. The pipeline's coefficients are `px = [slope intercept]` and `py = [slope intercept]`, so the mapping is:

- `trackVx = px(1)` and `trackX0 = px(2)`
- `trackVy = py(1)` and `trackY0 = py(2)`
- `trackT0 = 0`

This only works if the time zero in this script matches the pipeline's. The pipeline's own time axis may start somewhere other than the file's first event. In the pipeline plot you shared, the synthetic streak ran from about 0.4 s to 2.5 s, so the pipeline's time axis does not start where the simulator's does. Check how your loader sets time zero and adjust `tZero_us` or `trackT0` until the track lines up. You can confirm alignment in the windowed snapshot (Figure 1): the black cross should sit on the target.

The pipeline currently returns velocities but not the fit coefficients, so add `results.px = px; results.py = py;` to `analyze_angular_rate` to read them out.

Coordinates must also match. Pixels are 1-based in MATLAB; set `zeroIndexed` correctly for the file.

5. What the script computes

1. **Load.** Reads the file, converts to 1-based pixels, converts 0/1 polarity to -1/+1, converts time to seconds relative to `tZero_us`, and sorts by time.
2. **Geometry.** For every event it finds where the track says the target was at that instant, then splits the event's offset into an along-track part (positive ahead of the target, negative behind) and a cross-track part (sideways from the path).
3. **Gate.** Events inside the along-track and cross-track ranges are treated as target-driven.
4. **Background.** Counts events in every pixel farther than `bgExclude` from the track line and reports the rate, the busiest pixel, the fraction of pixels that fired at all, and the share of positive events.
5. **Target.** Counts events in the gate, converts to a rate over the time the track is on the sensor, and estimates how many gate events are actually noise.
6. **Shape.** For positive and negative events separately, reports the median and 10th/90th percentile of the along-track offset and the rms cross-track spread.

6. Console summary reference

Printed line	What it means	What it calibrates
Events, duration, overall rate	Size of the recording	Sanity check
Overall polarity	Share of + events across the file	Compare real against synthetic
Background mean rate	Mean events/s/pixel away from the track	`noiseRate` (clean mode), or the target for tuning `lambda_r_full`
Mean, top 1% removed	The same, ignoring the busiest 1% of pixels	Removes hot pixels and stars. Reads low when noise is very sparse.
Busiest pixel	Highest single-pixel rate	Shows whether hot pixels exist (the simulator does not model them)
Pixels with at least 1 event	Fraction of background pixels that fired	Checks that the noise spatial spread matches

Printed line	What it means	What it calibrates
Positive fraction (background)	+ share of background events	Whether the simulator's 50/50 assumption holds
Track speed, time on sensor	From the track you supplied	Consistency check
Events in gate (rate)	Target-driven event count and rate	`Phi_0` and `a`
Noise estimate in gate	Expected noise events inside the gate	How much the gate counts are contaminated
+ events: median, 10/90, rms cross	Where positive events land	Centroid alignment and blur width
- events: median, 10/90, rms cross	Where negative events land	Tail length, which depends mainly on `a`

7. Figures

- **Figure 1, windowed snapshot.** Net events (+1 and -1 summed) per pixel in a short window at time `tc`. Red is net positive, blue net negative. The black cross is where the track says the target is. If it is not on the target, the track or time zero is wrong.
- **Figure 2, event rate vs time.** Events per second split by polarity. The flat floor is background; anything above it while the target is on the sensor is the target. Compare the floor and the plateau between real and synthetic.
- **Figure 3, offsets from the track.** Left: each gate event's along-track offset against time (blue is -, red is +, thinned to 20,000 points). Right: histograms of the offsets. Positive events cluster at zero. Negative events trail behind, and the width of that histogram is the tail.

8. Worked example

The table below is from a Python run of the same calculations on a simulated version of the default scene (clean mode, noise at 0.1 events/s/pixel, track from the simulator's defaults). It is not a MATLAB run, and MATLAB values will differ a little because of random seeds.

Quantity	Result
Duration, total events	2.09 s, 248,287
Background mean rate	0.100 events/s/pixel
Background mean, top 1% removed	0.091 events/s/pixel
Pixels with at least 1 event	18.9%
Background positive fraction	50.1%
Track on sensor	2.09 s
Events in gate	57,300 (about 27,400 events/s)
Estimated noise in gate	about 2,070 (3.6%)
+ events	20,501; median -0.2 px (10/90: -2.8 / +2.4); rms cross 2.57 px
- events	36,799; median -49.7 px (10/90: -143.0 / -14.1); rms cross 2.54 px

The injected noise rate of 0.1 events/s/pixel is recovered exactly, which confirms the background measurement.

9. Assumptions

9.1 Qualitative assumptions

- **The track is supplied by you.** It is a straight line at constant velocity. The script never detects the target, so a wrong track gives wrong statistics without any error message.
- **Time and indexing are consistent.** The file's time zero and pixel indexing match the settings (`tZero_us` , `trackT0` , `zeroIndexed`) and the time base of the track.
- **Target events are identified by position alone.** Any event inside the gate counts, whether or not it came from the target.
- **Everything outside `bgExclude` is background.** That includes stars, hot pixels and other satellites.

- **Background is stationary and uniform.** The rate measured away from the track is assumed to hold over the whole recording and inside the gate. The noise estimate in the gate rests on this.
- **The whole decay tail lies inside the along-track gate.** Events beyond it are ignored.
- **Polarity is 0/1 or -1/+1.** A file with no negative values is treated as 0/1.
- **The target stays on the sensor during the pass.** The on-sensor time uses the track centerline only, not the blur margin.
- **The same script and settings run on real and synthetic files,** so any bias in the method cancels in the comparison.

9.2 Quantitative assumptions

Setting	Value	Basis
Sensor size	1280 x 720 px	EVK3 HD; change `nx` , `ny` for other cameras
`alongRange`	-300 to +30 px	Chosen to cover the tail seen in the simulation (- event 10th percentile -143 px at 614 px/s with a = 10). A longer tail needs a wider range.
`crossGate`	15 px either side	About 6 times the cross-track rms of about 2.5 px measured in the simulation
`bgExclude`	40 px	Chosen so the blur and tail do not contaminate the background
Hot-pixel trim	top 1% of background pixels	Chosen
Gate area	(30 + 300) x (2 x 15) = 9,900 px at the defaults	Used for the noise estimate: background rate x area x duration
Time bins	200 over the file (10.45 ms each for 2.09 s)	Chosen
Snapshot window	5 ms, color range plus or minus 10 net events	Chosen
Scatter thinning	about 20,000 points; histogram bin width 3 px	Plotting speed and readability

Setting	Value	Basis
On-sensor time	Track sampled at 10,001 points, so resolution is duration / 10,000	Chosen
Percentiles	10, 50, 90 from the sorted offsets by index, with no interpolation	Implementation detail

10. Limitations

- The track is supplied by you and assumed to be a constant-velocity straight line. A curved or accelerating track will give a misleading gate and offsets.
- Anything in the real data that is not the target (stars, hot pixels, other satellites) is counted as background unless it lies within `bgExclude` of the track. Compare the plain mean with the top-1%-removed mean to see how much they matter.
- The trimmed mean reads low for very sparse noise, because the busiest 1% of pixels then include ordinary noise pixels (0.091 versus the true 0.100 in the example).
- The cross-track rms is not the same as the blur sigma. In the example it was about 2.5 px for a sigma of 1.5 px, because events also fire in the blur's wings. Compare real and synthetic values rather than reading it as sigma.
- The along-track gate extends well behind the target. If the real tail is much shorter, narrow `alongRange` to avoid counting extra noise.
- Real data will include any edge-clipping and fit bias in the pipeline's track, which shifts the offsets slightly.

11. Troubleshooting

Symptom	Likely cause and fix
Error: events fall outside the sensor	Wrong `nx`, `ny` or `zeroIndexed`.
Warning: no events in the snapshot window	`tc` is outside the recording or the target is not on the sensor then. Change `tc`.
Snapshot cross is not on the target	The track values or time zero are wrong. Check `tZero_us`, `trackT0` and the pipeline's time origin.

Symptom	Likely cause and fix
Gate count is tiny	Track misaligned, gate too narrow, or the target is not on the sensor.
`readmatrix` not found	MATLAB older than R2019a. Use `dlmread` or `importdata`.
Background rate looks too high	Stars or hot pixels in the real data. Use the top-1%-removed value or raise `bgExclude`.